

Keras

ЛЕКЦИЯ №4



- 1. Схема обучения нейросети.**
- 2. Примеры**

Схема обучения нейросети.

Шаг 1. Сбор датасета.

Шаг 2. Создание обучающей и тестовой выборок: обучающий, проверочный (валидационный) и тестовый датасеты.

Шаг 3. Определение архитектуры Keras, настройка гиперпараметров.

Шаг 4. Компиляция модели

Шаг 5. Обучение, отслеживание оценки качества НС работы на обучающей и проверочной выборке. Нередки ситуации, когда на обучающей выборке будет 100%, но важна точность именно на проверочной.

Шаг 6. Если результаты предыдущего шага не соответствуют (слишком низкая точность или началось переобучение, см. далее), изменение гиперпараметров, добавление при необходимости слоев нормализации и регуляризации.

Шаг 7. Оценка точности работы сети на тестовом наборе.

Написание нейронных сетей с библиотекой Tensorflow.

На данный момент библиотека Tensorflow стала настолько большой, что переросла в экосистему.

Сервисы

- интеграция нейросетей (TF Serving),
- хранение моделей в облачных хранилищах (TensorFlow Hub),
- API по распознаванию объектов (Tensorflow Object Detection API).
- написание и обучение нейросетей на глубоком уровне.

Основное отличие от верхнеуровневого использования - это возможность модификации стандартных методов и функции, возможность создавать свои подходы и решения к задачам.

Написание нейронных сетей с библиотекой Tensorflow.

Основной тип данных для работы называется `tf.Tensor`.
Библиотека импортируется стандартной записью.

```
import tensorflow as tf

# Создаем тензор
# Мы можем присвоить ему тип данных, например, float16
# Если этого не сделать, по умолчанию для целых чисел
# будет использоваться int32

rank_0_tensor = tf.constant(4, dtype=tf.float16)

print('Это тензор, созданный нами. Вся информация о нем, в
том числе и тип данных: ')
print(rank_0_tensor)
```

Это тензор, созданный нами. Вся информация о нем, в том
числе и тип данных:

```
tf.Tensor(4.0, shape=(), dtype=float16)
```

Формы тензоров.

- **Shape:** Количество элементов по каждому измерению тензора
- **Rank:** Номер измерения тензора. Для числа rank = 0, для вектора rank = 1, а для матрицы rank = 2.
- **Axis** или **Dimension:** Измерения тензоров
- **Size:** Общее число элементов в тензоре. Вектор формы

```
[ ] print("Тензор:", rank_3_tensor, "\n")

print("Число измерений:", rank_3_tensor.ndim)
print("Тип каждого элемента:", rank_3_tensor.dtype)
print("Форма тензора:", rank_3_tensor.shape)
print("Элементы тензора на оси с индексом -1:", rank_3_tensor[0])
print("Элементы тензора на оси с индексом -1:", rank_3_tensor[-1])
```

```
Тензор: tf.Tensor(
[[[2. 7.]
  [8. 5.]]

 [[3. 1.]
  [4. 9.]]
```

Нейронные сети с Tensorflow.

С тензорами возможны все стандартные математические действия: сложение, умножение, возведение в степень, матричное умножение. Примеры таких действий и результат записаны в коде:

```
print(tf.add(1, 2)) # Сложение
tf.Tensor(3, shape=(), dtype=int32)
```

```
print(tf.add([1, 2], [3, 4])) # Сложить по элементам
tf.Tensor([4 6], shape=(2,), dtype=int32)
```

```
print(tf.square(5)) # Возведение в квадрат
tf.Tensor(25, shape=(), dtype=int32)
```

```
print(tf.reduce_sum([[1, 2, 3], [4, 5, 6]])) # Сложение
всех элементов
tf.Tensor(21, shape=(), dtype=int32)
```

```
print(tf.square(2) + tf.square(3)) # Перегрузка
операторов
tf.Tensor(13, shape=(), dtype=int32)
```

Нейронные сети с Tensorflow.

Совместимость с NumPy

Работа с массивами в TensorFlow хорошо совместима с NumPy ndarray:

TensorFlow операции автоматически конвертируют NumPy массивы в тензоры.

NumPy операции автоматически конвертируют Tensors в NumPy массивы.

```
print('Один и тот же тензор, преобразованный в NumPy  
разными методами')  
print(np.array(rank_3_tensor))  
print('\n')  
print(rank_3_tensor.numpy())
```


Нейронные сети с Tensorflow.

Broadcasting (пакетные вычисления) - это полный набор операций над тензорами, заимствованный из NumPy, с поддержкой матричных вычислений над массивами разной формы и конвертирования между этими формами. Самый простой и наиболее распространенный случай - умножить или добавить тензор в скаляр. В этом случае скаляр передается в той же форме, что и другой аргумент.

Аналогично одномерные могут быть растянуты, чтобы соответствовать другим аргументам. Оба аргумента могут быть растянуты в одном и том же вычислении

```
x = tf.constant([1, 2, 3])
y = tf.constant(2)
z = tf.constant([2, 2, 2])
# Разные методы одного и того же вычисления
print(tf.multiply(x, 2))
print(x * y)
print(x * z)

tf.Tensor([2 4 6], shape=(3,), dtype=int32)
tf.Tensor([2 4 6], shape=(3,), dtype=int32)
tf.Tensor([2 4 6], shape=(3,), dtype=int32)
```

Строковый тензор

Как и NumPy, `tf.string` является `dtype`, можно представлять данные в виде строк (байтовых массивов переменной длины) в тензорных числах.

Строки являются атомарными и не могут быть проиндексированы, как строки Python.

Длина строки не является одним из измерений тензора.

Для управления - `tf.strings`

НЕЛЬЗЯ использовать `tf.cast`, чтобы превратить строковый тензор в числовой, но можно преобразовать его в байты, а затем в числа.

Тип `tf.string dtype` используется для всех необработанных байтовых данных в TensorFlow.

```
[ ] stroka = 'Демо-строка'
    stroka[5]
```

```
'c'
```

```
[ ] # Тензоры могут быть строками
    string_tensor = tf.constant("Gray wolf")
    print(string_tensor)
```

```
tf.Tensor(b'Gray wolf', shape=(), dtype=string)
```

```
[ ] print(string_tensor[5])
```

GPU и размещение на устройстве

Проверка доступных списков устройств и размещение данных на конкретном модуле.

```
print("Доступные модули: "),  
print(tf.config.experimental.list_physical_devices())  
  
print("Доступные модули GPU: "),  
print(tf.config.experimental.list_physical_devices("GPU"))
```

GPU и размещение на устройстве

Раздел `tf.data.Dataset` API представляет готовый конвейер для обучения моделей:

- `tf.data.Dataset.from_tensor_slices()` - данные из массивов
- `tf.data.Dataset.from_generator()` - данные через генераторы
- `tf.data.TFRecordDataset()` - специальный формат данных для ускорения работы
- `tf.data.TextLineDataset()` - данные из текстовых файлов
- `tf.data.Dataset.list_files()` - список файлов с данными

Keras

Keras — это библиотека для языка программирования Python, которая предназначена для глубокого машинного обучения. Она позволяет быстрее создавать и настраивать модели (надстройка над другими библиотеками).

- Удобное построение моделей, по которым будет проходить обучение.
- Настройка слоев в моделях — обычно подбор нужного количества слоев требуется для точности.
- Обработка ввода и вывода информации из модели.
- Преобразование входных данных, которые поступают в обучаемую модель.
- Удобный подбор датасетов для обучения.
- Визуализация модели.
- Подготовка модели к работе, определение ее функций ошибки и оптимизаторов.
- Обучение и тестирование модели.
- Сборка и первичный запуск программы машинного обучения.



Шаг 1. Загрузка датасета. Keras

```
from tensorflow.keras.datasets import mnist # загрузка всего датасет MNIST  
(x, y), (_, _) = mnist.load_data() # запись в переменные нужных нам частей  
# Посмотрим на размерность данных
```

```
print(x.shape)  
print(y.shape)
```



Шаг 2. Создание обучающей и тестовой выборки

Обучающая выборка (training set, x_{train}) - используется для обучения сети.

Проверочная выборка (validation set, x_{val}) - используется в процессе обучения для оценки качества обучения.

Использование проверочной выборки – первый шаг к оценке качества работы НС.

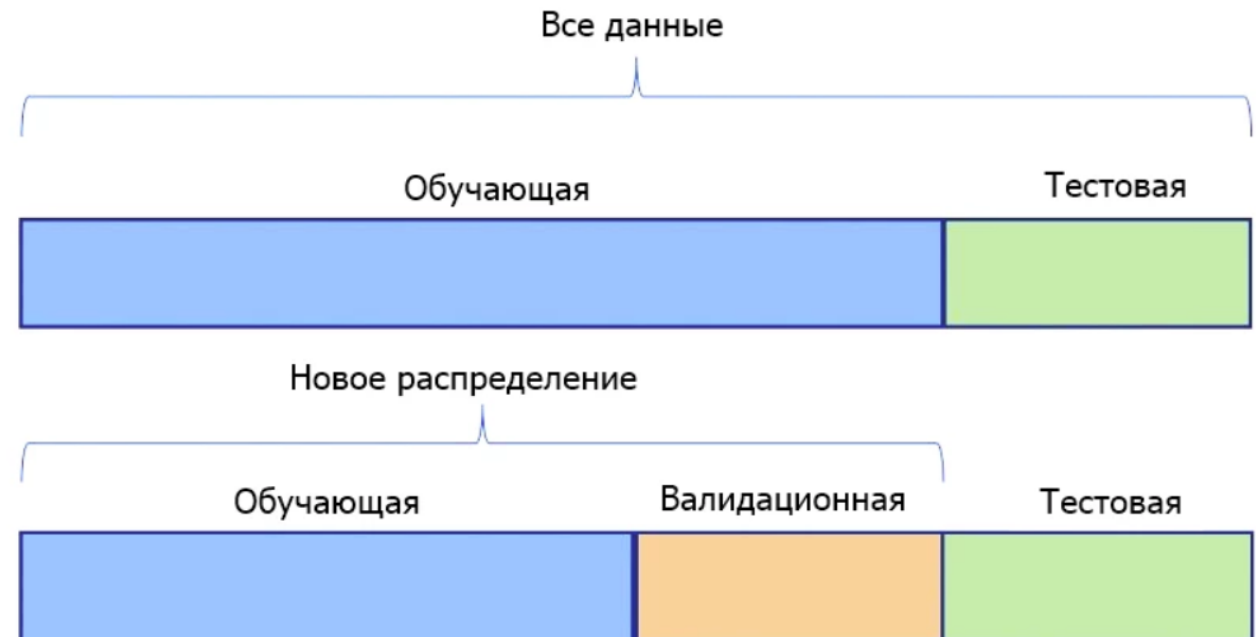
Этот набор используется на каждой эпохе обучения.

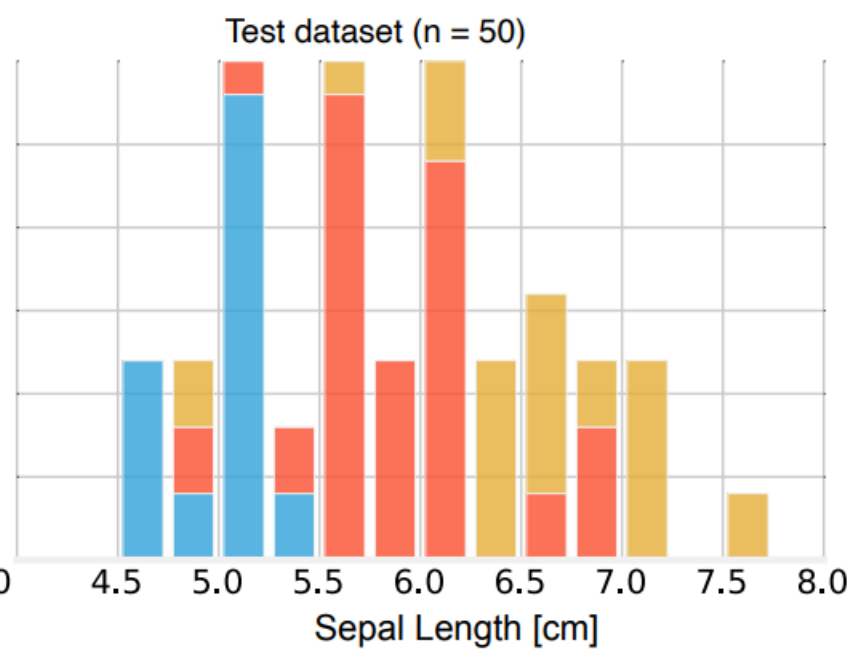
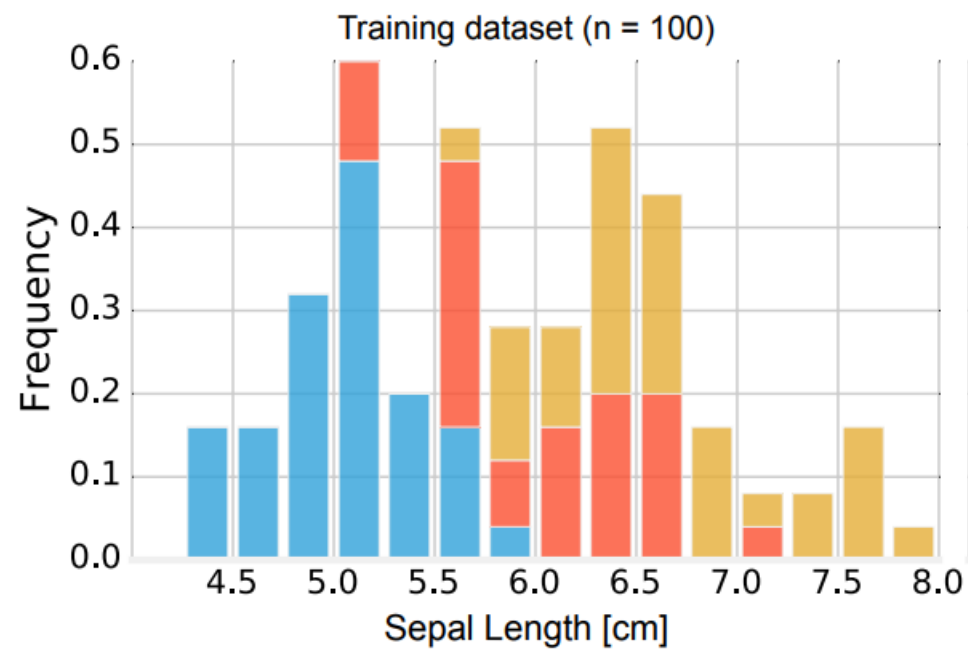
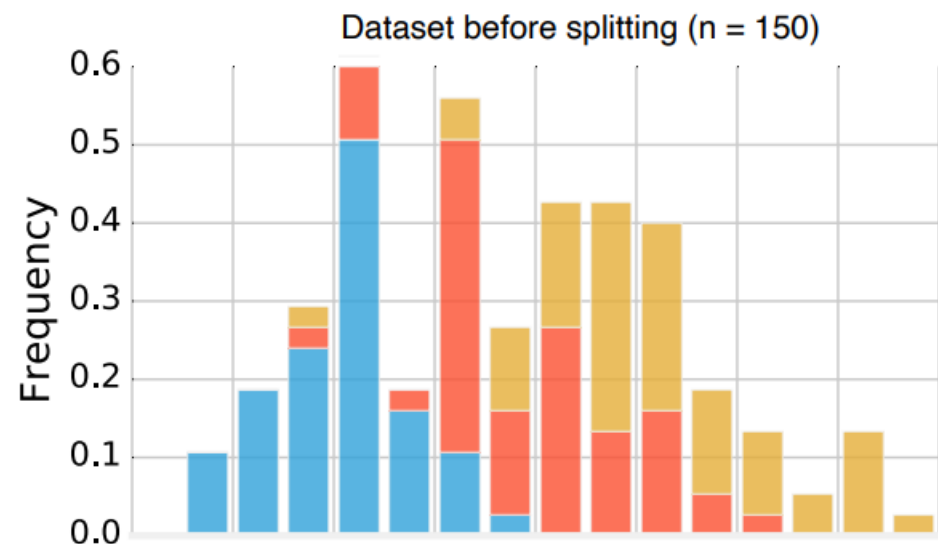
Важно, чтобы данные из проверочной и обучающей выборок не пересекались и не повторялись.

Вычленение проверочной части происходит обычно при подаче выборки на обучение, внутри метода `.fit()`,

Можно выделить часть списка при помощи слайсинга, либо через параметр метода `validation_split=0.3`.

Тестовая выборка (test set, x_{test}) - используется для оценки качества работы сети после завершения обучения. Чаще всего ее пропорция следующая: 80% - 15% - 5% или 80% - 10% - 10%.





Шаг 2. Создание обучающей и тестовой выборки

Самый простой, без использования сторонних функций. Позволяет сразу делить весь датасет на нужное количество выборок.

Минус способа - классы как были последовательно расположены, так и останутся, это снижает эффективность обучения.

Пропорции 80% - 10% - 10%:

обучающая выборка (x_{train}) составит $60000 * 0,8 = 48000$

валидационная выборка (x_{val}) составит $60000 * 0,1 = 6000$

тестовая выборка (x_{test}) составит $60000 * 0,1 = 6000$

```
x_train = x[:48000]
```

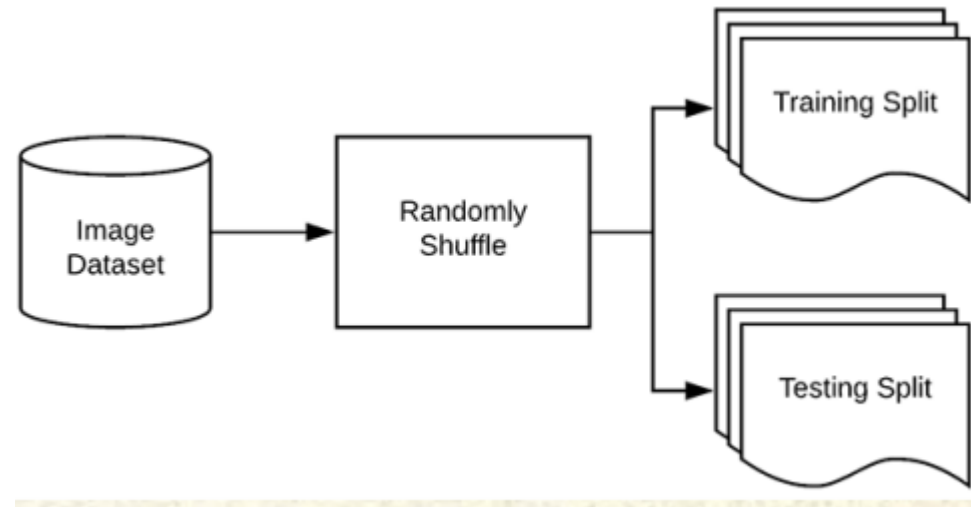
```
x_val = x[48000:54000]
```

```
x_test = x[54000:]
```

```
print('Обучающая выборка ', x_train.shape)
```

```
print('Валидационная выборка', x_val.shape)
```

```
print('Тестовая выборка ', x_test.shape)
```



Шаг 2. Создание обучающей и тестовой выборки

Библиотека sklearn содержит в себе функцию train_test_split

импорт библиотеки

```
from sklearn.model_selection import train_test_split
```

Функция имеет несколько аргументов:

x_data и y_data – разделяемые наборы данных;

test_size – процент значений, которые пойдут в тестовую выборку;

shuffle – требование перемешивания данных.

.

Функция возвращает:

x_train, x_test, y_train, y_test – результаты работы функции, обучающая и тестовая выборки;

```
x_train, x_test, y_train, y_test = train_test_split(x, # ----- датасет с изображениям
                                                    y, # ----- датасет с метками
                                                    test_size = 0.1, # --- процент тестовых значений
                                                    shuffle=True, # ----- перемешивание
                                                    random_state=42) # ---
```

```
print('Обучающая выборка изображений', len(x_train))
```

```
print('Обучающая выборка меток', len(y_train))
```

```
print()
```

```
print('Тестовая выборка изображений', len(x_test))
```

```
print('Тестовая выборка меток', len(y_test))
```

Шаг 2. Создание обучающей и тестовой выборки

Библиотека sklearn содержит в себе функцию train_test_split

импорт библиотеки

```
from sklearn.model_selection import train_test_split
```

Функция имеет несколько аргументов:

x_data и y_data – разделяемые наборы данных;

test_size – процент значений, которые пойдут в тестовую выборку;

shuffle – требование перемешивания данных.

.

Функция возвращает:

x_train, x_test, y_train, y_test – результаты работы функции, обучающая и тестовая выборки;

```
x_train, x_test, y_train, y_test = train_test_split(x, # ----- датасет с изображениям
                                                    y, # ----- датасет с метками
                                                    test_size = 0.1, # --- процент тестовых значений
                                                    shuffle=True, # ----- перемешивание
                                                    random_state=42) # ---
```

```
print('Обучающая выборка изображений', len(x_train))
```

```
print('Обучающая выборка меток', len(y_train))
```

```
print()
```

```
print('Тестовая выборка изображений', len(x_test))
```

```
print('Тестовая выборка меток', len(y_test))
```

Шаг 3. Подбор гиперпараметров

Подбор гиперпараметров

Каждая НС имеет два вида параметров: изменяемые и неизменяемые.

Изменяемые –

самой нейросетью от эпохи к эпохе заключается суть обучения, к ним относятся весовые коэффициенты.

Гиперпараметры модели.

- количество слоев в модели;
- типы слоев нейросети
- количество нейронов в каждом слое;
- скорость обучения (**learning rate**);
- размер мини-выборки при обучении (**batch size**).
- функция ошибки ...

Шаг 3. Подбор гиперпараметров

Подбор гиперпараметров

Каждая НС имеет два вида параметров: изменяемые и неизменяемые.

Изменяемые –

самой нейросетью от эпохи к эпохе заключается суть обучения, к ним относятся весовые коэффициенты.

Гиперпараметры модели.

- количество слоев в модели;
- типы слоев нейросети
- количество нейронов в каждом слое;
- скорость обучения (**learning rate**);
- размер мини-выборки при обучении (**batch size**).
- функция ошибки ...

Шаг 4. Создание модели сети

from tensorflow.keras.models import Sequential / Создание линейной модели

Dense-слой.

Полносвязный слой. Как механизм делится по слоям, так и некоторые слои тоже делятся на составляющие элементы.

```
model = Sequential()
from tensorflow.keras.layers import Dense
model.add(Dense(32, input_dim=10))
```

```
model = Sequential()
model.add(Dense(32, input_dim=10))
model.add(Dense(5))
model.add(Dense(1))
model.summary()
```

Model: "sequential_1"

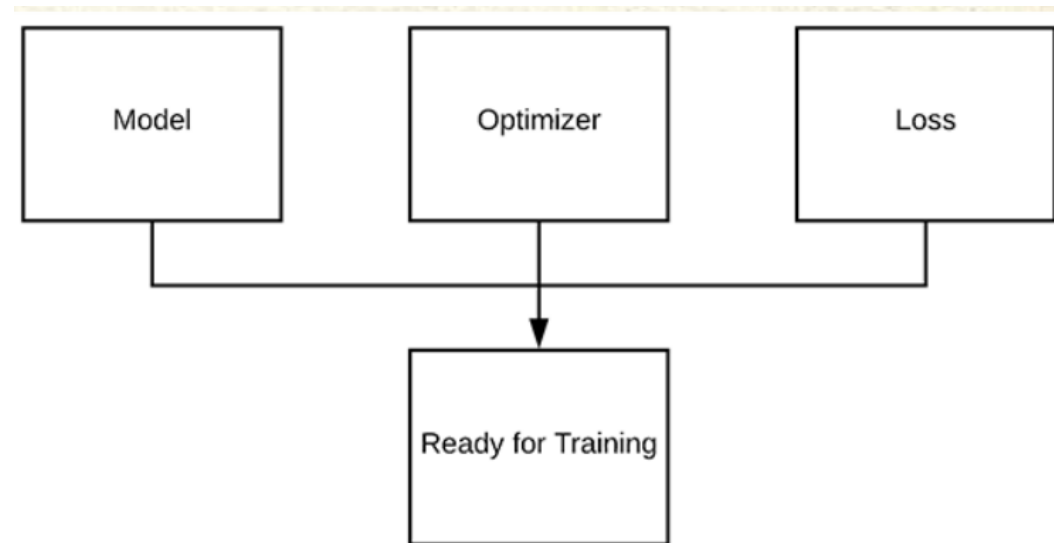
Layer (type)	Output Shape	<u>Param #</u>
=====		
=====		
dense_1 (Dense)	(None, 32)	352
dense_2 (Dense)	(None, 5)	165
dense_3 (Dense)	(None, 1)	6

=====

=====
Total params: 523
Trainable params: 523
Non-trainable params: 0

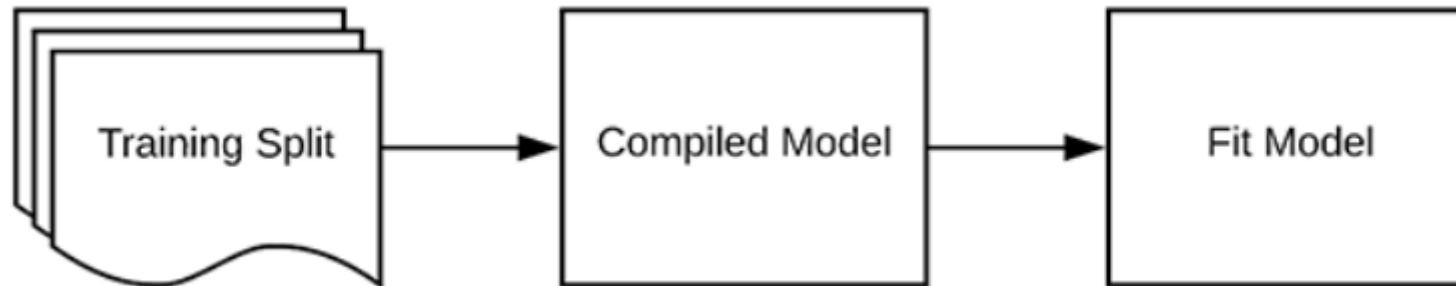
Шаг 5. Компиляция модели

```
model.compile(loss='categorical_crossentropy', optimizer='adam')
```



Шаг 6. Обучение модели

```
history = model.fit(x_train,          # Обучающая выборка  
                    y_train,          # Обучающая выборка меток класса  
                    batch_size=8,     # Размер батча (пакета)  
                    epochs=100,       # Количество эпох обучения  
                    validation_split=0.1, # доля проверочной выборки  
                    verbose=1)        # Отображение хода обучения
```



Визуализация обучения

```
from tensorflow.keras.models import Sequential
# Основные слои
from tensorflow.keras.layers import Dense

# Оптимизаторы
from tensorflow.keras.optimizers import Adam
# -----

from tensorflow.keras.datasets import mnist # загрузка всего датасет MNIST
(x_train_org, y_train_org), (x_test_org, y_test_org) = mnist.load_data()

x_train = x_train_org.reshape(x_train_org.shape[0], -1)
x_test = x_test_org.reshape(x_test_org.shape[0], -1)

x_train = x_train.astype('float32') / 255.
x_test = x_test.astype('float32') / 255.

from tensorflow.keras import utils
y_train = utils.to_categorical(y_train_org, 10)
y_test = utils.to_categorical(y_test_org, 10)
```

Визуализация обучения

Создание модели

```
model = Sequential()
```

Добавление слоев

```
model.add(Dense(800, input_dim=784, activation='relu'))
```

```
model.add(Dense(400, activation='relu'))
```

```
model.add(Dense(10, activation='softmax'))
```

Компиляция и возврат модели

```
model.compile(loss='categorical_crossentropy',  
              optimizer=Adam(learning_rate=0.001),  
              metrics=['accuracy'])
```

Результат обучения сети, (то есть вместо model.fit() запишем history = model.fit():

```
history = model.fit(x_train,          # Обучающая выборка  
                    y_train,         # Обучающая выборка меток класса  
                    batch_size=256,  # Размер батча (пакета)  
                    epochs=100,      # Количество эпох обучения  
                    validation_split=0.1, # доля проверочной выборки  
                    verbose=1)       # Отображение хода обучения
```

```
print(type(history.history))
```

Визуализация обучения

Вывод параметров

```
print(history.history.keys())
```

```
dict_keys(['loss', 'accuracy', 'val_loss', 'val_accuracy'])
```

'loss' - значения ошибки на обучающей выборке

'accuracy' - точность на обучающей выборке

'val_loss' - значения ошибки на проверочной выборке

'val_accuracy' - точность на проверочной выборке

Вывод значений ошибки (loss) на обучающей выборке

```
print(history.history['loss'][:5])
```

Вывод значений ошибки (loss) на обучающей выборке

```
print(history.history['loss'][:5])
```

Визуализация обучения

```
history = model.fit(x_train,      # Обучающая выборка
                    y_train,      # Обучающая выборка меток класса
                    batch_size=8, # Размер батча (пакета)
                    epochs=100,   # Количество эпох обучения
                    validation_split=0.1, # доля проверочной выборки
                    verbose=1)    # Отображение хода обучения
```

Epoch 1/100

6750/6750 [=====] - 15s 2ms/step - loss: 0.1991 - accuracy: 0.9393 - val_loss: 0.1002
- val_accuracy: 0.9713

Epoch 2/100

6750/6750 [=====] - 13s 2ms/step - loss: 0.1006 - accuracy: 0.9701 - val_loss: 0.0795
- val_accuracy: 0.9767

Epoch 3/100

6750/6750 [=====] - 13s 2ms/step - loss: 0.0753 - accuracy: 0.9781 - val_loss: 0.1040
- val_accuracy: 0.9708

Epoch 4/100

6750/6750 [=====] - 13s 2ms/step - loss: 0.0631 - accuracy: 0.9820 - val_loss: 0.1066

Визуализация обучения

Рисование графиков

```
import matplotlib.pyplot as plt
```

Отрисовка графика точности на обучающей выборке

```
plt.plot(history.history['accuracy'], label='Доля верных ответов на обучающем наборе')
```

Отрисовка графика точности на проверочной выборке

```
plt.plot(history.history['val_accuracy'], label='Доля верных ответов на проверочном наборе')
```

```
plt.xlabel('Эпоха обучения')
```

```
plt.ylabel('Доля верных ответов')
```

```
plt.legend()
```

```
plt.show()
```

Построим графики ошибки:

```
plt.plot(history.history['loss'],
```

```
        label='Ошибка на обучающем наборе')
```

```
plt.plot(history.history['val_loss'],
```

```
        label='Ошибка на проверочном наборе')
```

```
plt.xlabel('Эпоха обучения')
```

```
plt.ylabel('Ошибка')
```

```
plt.legend()
```

```
plt.show()
```

Визуализация обучения

Рисование графиков

```
import matplotlib.pyplot as plt
```

Отрисовка графика точности на обучающей выборке

```
plt.plot(history.history['accuracy'], label='Доля верных ответов на обучающем наборе')
```

Отрисовка графика точности на проверочной выборке

```
plt.plot(history.history['val_accuracy'], label='Доля верных ответов на проверочном наборе')
```

```
plt.xlabel('Эпоха обучения')
```

```
plt.ylabel('Доля верных ответов')
```

```
plt.legend()
```

```
plt.show()
```

Построим графики ошибки:

```
plt.plot(history.history['loss'],
```

```
        label='Ошибка на обучающем наборе')
```

```
plt.plot(history.history['val_loss'],
```

```
        label='Ошибка на проверочном наборе')
```

```
plt.xlabel('Эпоха обучения')
```

```
plt.ylabel('Ошибка')
```

```
plt.legend()
```

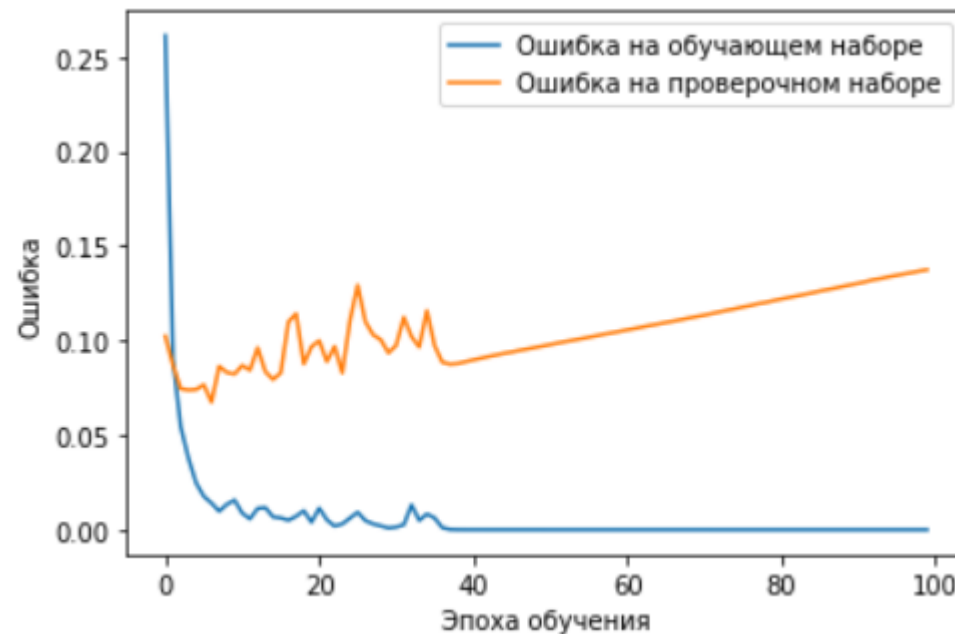
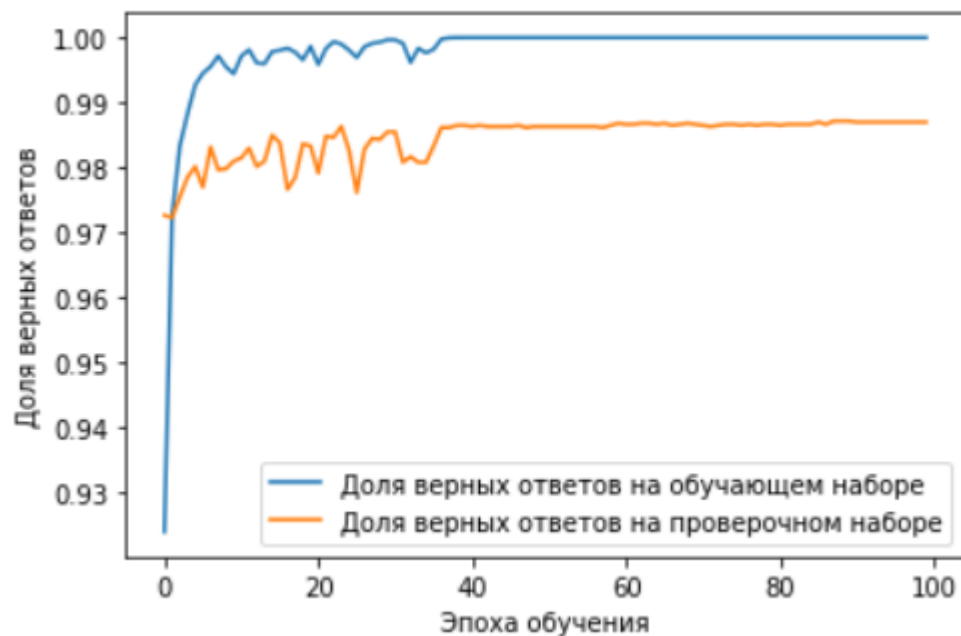
```
plt.show()
```

Визуализация обучения

```
# Отрисовка графика точности на обучающей выборке  
plt.plot(history.history['accuracy'], label='Доля верных ответов на обучающем наборе')  
# Отрисовка графика точности на проверочной выборке
```

Построим графики ошибки:

```
plt.plot(history.history['loss'],  
         label='Ошибка на обучающем наборе')
```



Activation – слой

Задача бинарной классификации (R – скала, M – мина).

Задача в том, чтобы научить нейросеть определять по параметрам, скала это или мина.

```
import numpy as np
import pandas as pd
# Последовательная модель НС
from tensorflow.keras.models import Sequential
# Основные слои
from tensorflow.keras.layers import Dense, Dropout, BatchNormalization
# Слой задания активационной функции <----- !!!
from tensorflow.keras.layers import Activation
# Утилиты предобработки данных
from tensorflow.keras import utils
# Оптимизаторы
from tensorflow.keras.optimizers import Adam
# Разделение на обучающую и проверочную/тестовую выборку
from sklearn.model_selection import train_test_split
# Рисование графиков
import matplotlib.pyplot as plt
%matplotlib inline

df = pd.read_csv("sonar.csv", header=None)

print(df.shape)
```


Activation – слой

Парсинг.

```
dataset = df.replace('R', 1.).replace('M', 0.).astype(float).to_numpy()
```

```
x_data = dataset[:, :60]
```

```
y_data = dataset[:, 60]
```

```
x_train, x_test, y_train, y_test = train_test_split(x_data, # набор параметров  
                                                    y_data, # набор меток классов  
                                                    test_size=0.2, # процент в тестовую  
                                                    shuffle=True, # перемешивание  
                                                    random_state=3) # воспроизводимость
```

```
# Выведем размерность полученных выборок
```

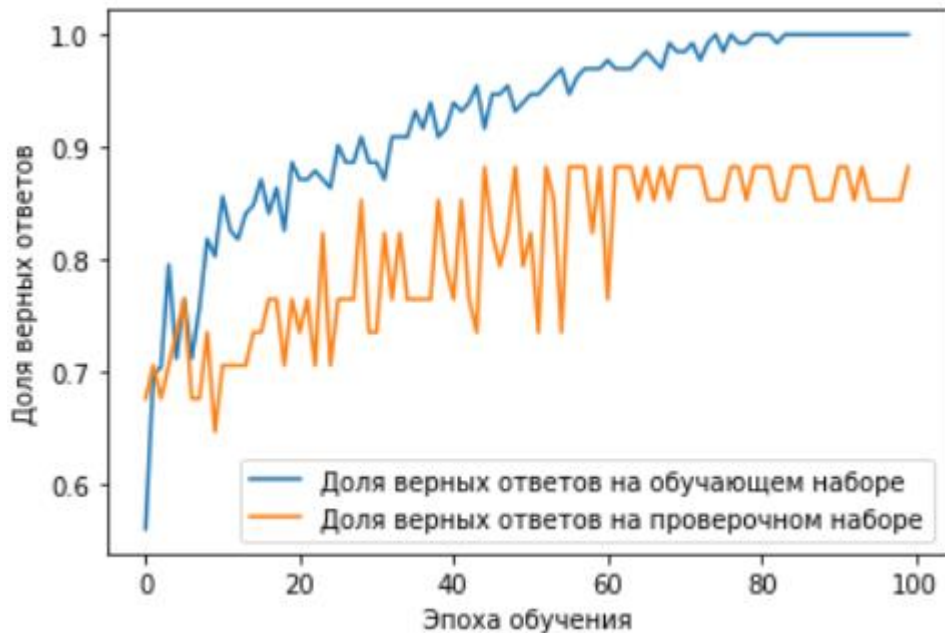
```
print('Обучающая выборка параметров', x_train.shape)  
print('Обучающая выборка меток классов', y_train.shape)  
print()  
print('Тестовая выборка параметров', x_test.shape)  
print('Тестовая выборка меток классов', y_test.shape)
```

Activation – слой

```
# Создание необученной модели при помощи функции create_model()  
model = create_model()
```

Обучение модели

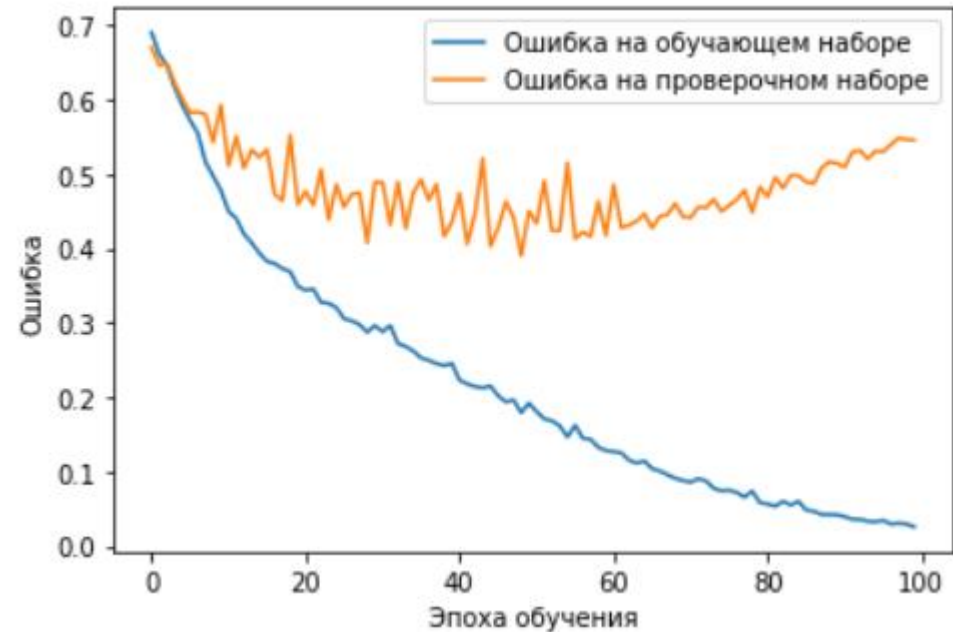
```
history = model.fit(x_train,      # Обучающая выборка параметров  
                    y_train,      # Обучающая выборка меток класса  
                    batch_size=8, # Размер батча (пакета)  
                    epochs=100,   # Количество эпох обучения  
                    verbose=1)    # Отображение хода обучения
```



```
# Создание необученной модели при помощи функции create_model()  
model = create_model()
```

Обучение нейронной сети

```
history = model.fit(x_train,      # Обучающая выборка параметров  
                    y_train,      # Обучающая выборка меток класса  
                    batch_size=8, # Размер батча (пакета)  
                    epochs=100,   # Количество эпох обучения  
                    validation_split=0.2, # Доля проверочной выборки  
                    verbose=1)    # Отображение хода обучения
```



Слой Dropout

```
# Создание последовательной модели
model = Sequential()
model.add(Dropout(0.3, input_shape=(x_train.shape[1],)))
model.add(Dense(60, activation='relu'))
model.add(Dropout(0.3))
model.add(Dense(30, activation='relu'))
model.add(Dropout(0.3))
model.add(Dense(1, activation='sigmoid'))
```

```
# Компиляция модели
model.compile(loss='binary_crossentropy',
              optimizer=Adam(learning_rate=0.001),
              metrics=['accuracy'])
```

```
# Обучение сети
history = model.fit(x_train,
                    y_train,
                    batch_size=8,
                    epochs=200,
                    validation_split=0.2,
                    verbose=1)
```

```
# Краткая сводка архитектуры модели
```

```
model.summary()
```

Model: "sequential_3"

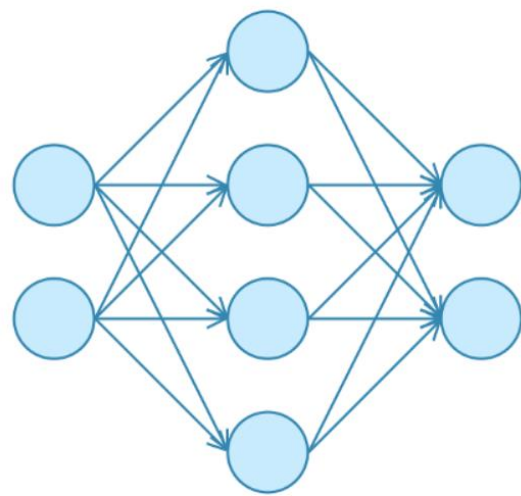
Layer (type)	Output Shape	Param #
dropout (Dropout)	(None, 60)	0
dense_9 (Dense)	(None, 60)	3660
dropout_1 (Dropout)	(None, 60)	0
dense_10 (Dense)	(None, 30)	1830
dropout_2 (Dropout)	(None, 30)	0
dense_11 (Dense)	(None, 1)	31

Total params: 5,521

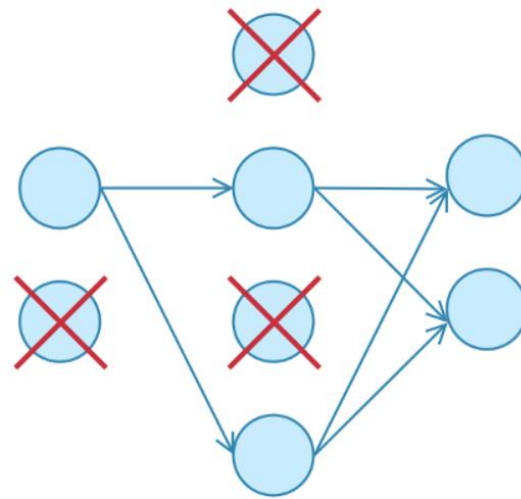
Trainable params: 5,521

Non-trainable params: 0

Слой Dropout

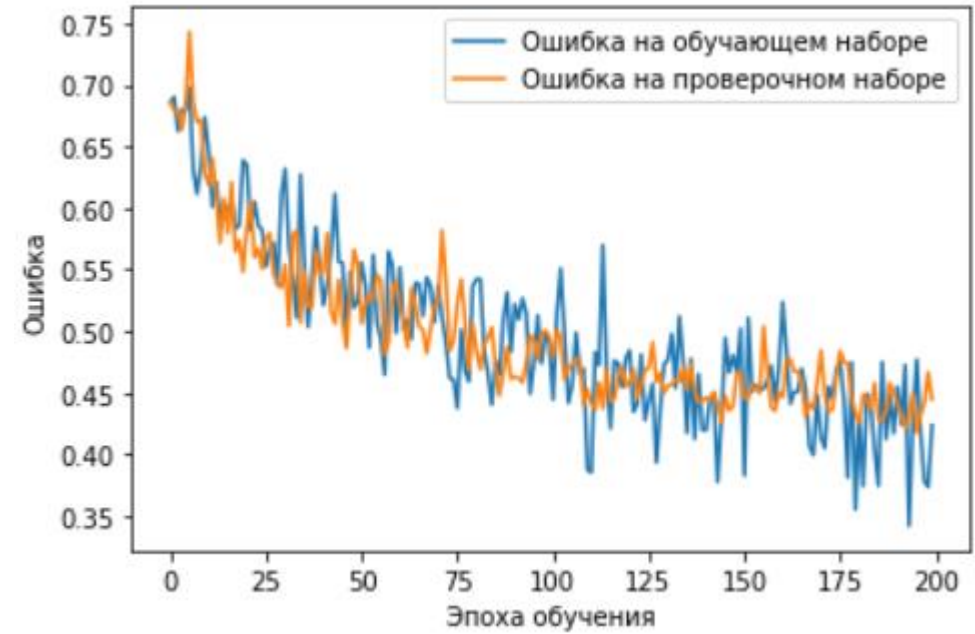
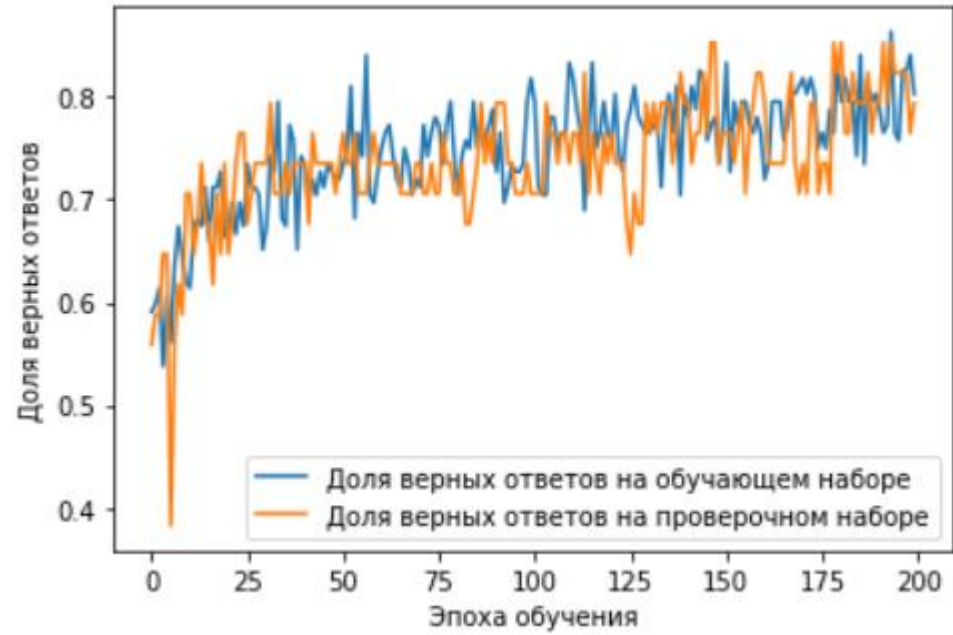


No Dropout

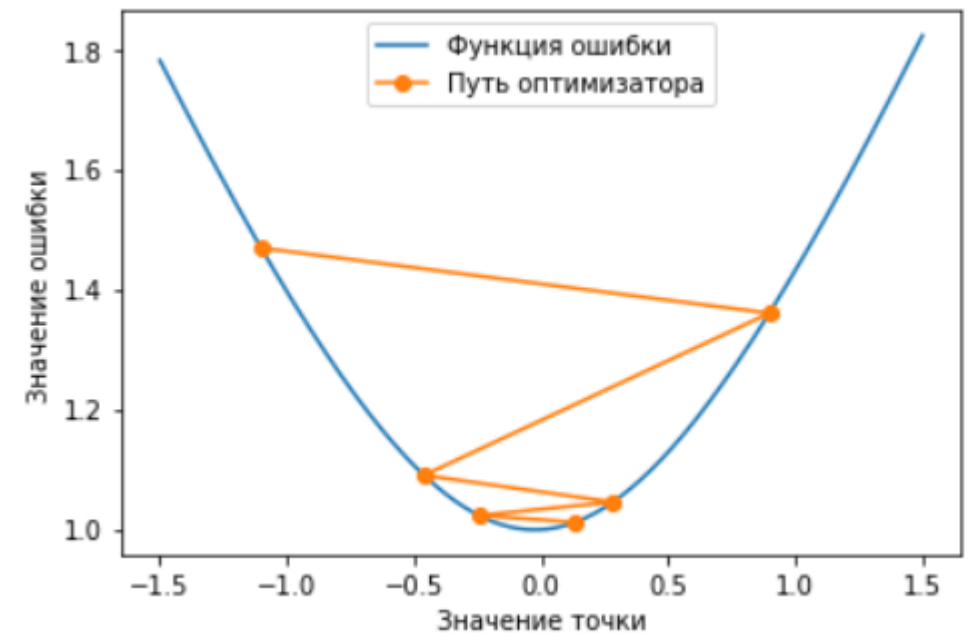
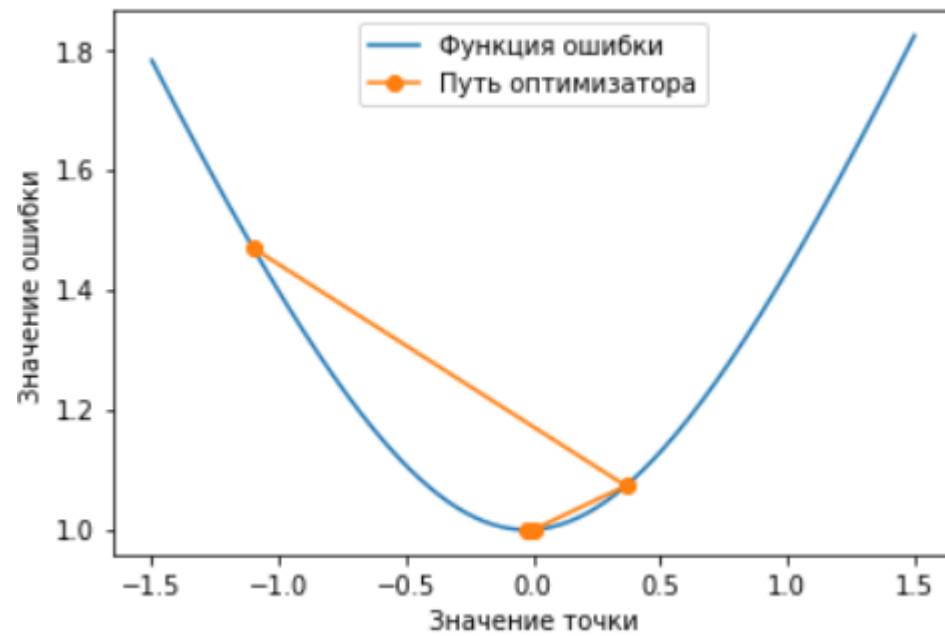


With Dropout

Слой Dropout

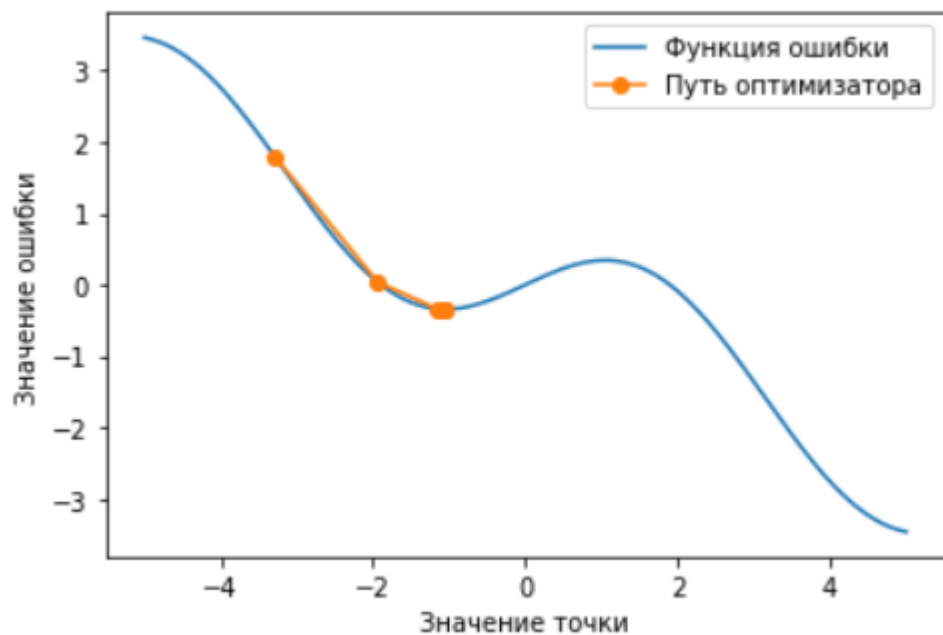


Adagrad и RMSprop



Momentum

$$\Delta w_{i,j}(t) := \alpha \cdot \Delta w_{i,j}(t - 1) + \eta \cdot \frac{\partial \mathcal{L}}{\partial w_{i,j}}(t)$$



$$w_{i,j}(t + 1) := w_{i,j}(t) - \Delta w_{i,j}(t)$$